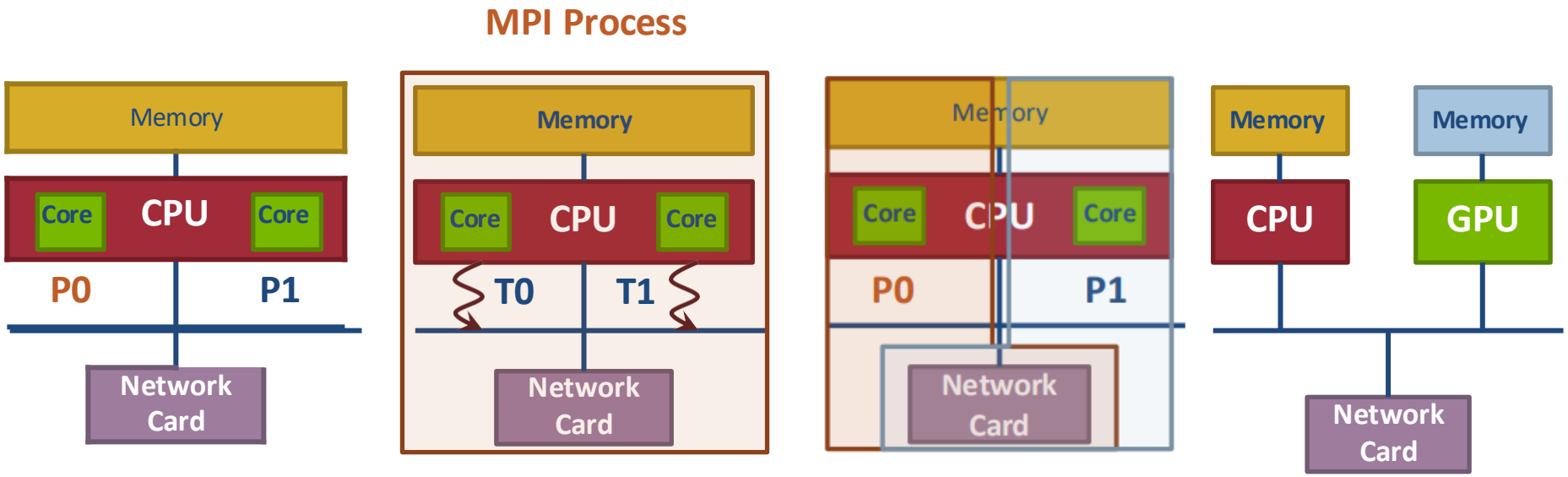
Introduction to Parallel Programming

Lecture xx: MPI + X

谭光明
高性能计算机研究中心

Hybrid MPI + X : Most Popular Forms

MPI + X



Flat MPI

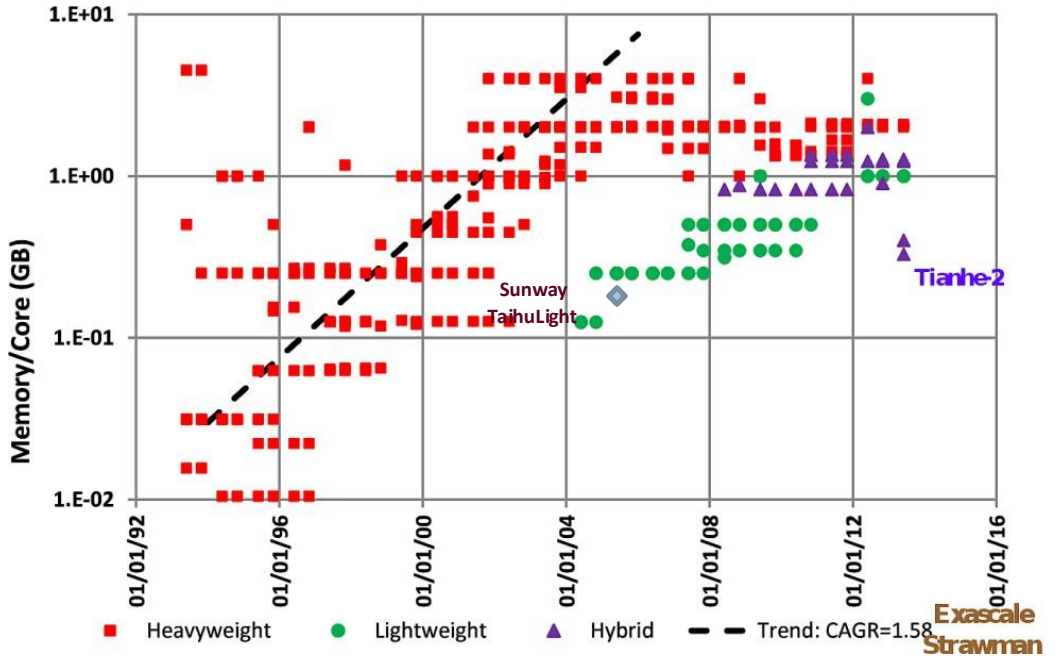
MPI + Threads

MPI + Shared Memory

MPI + ACC

Why Hybrid MPI+X? Towards Strong Scaling (1/3)

- Strong scaling applications is increasing in importance
  - Hardware limitations: not all resources scale at the same rate as cores (e.g., memory capacity, network resources)
  - Desire to solve the same problem faster on a bigger machine
    - Nek5000, HACC, LAMMPS
- Strong scaling pure MPI applications is getting harder
  - On-node communication is costly compared to load/stores
  - O(Px) communication patterns (e.g., All-to-all) costly



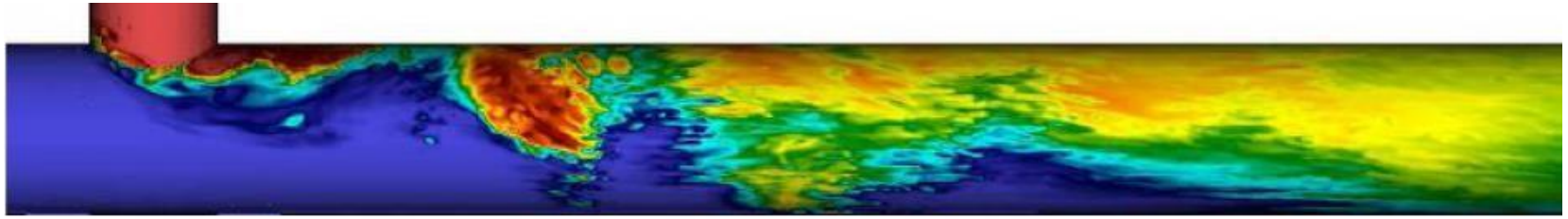
Evolution of the memory capacity per core in the Top500 list (Peter Kogge. PIM & memory: The need for a revolution in architecture.)

Why Hybrid MPI+X? Towards Strong Scaling (2/3)

- MPI+X benefits (X= {threads,MPI shared-memory, etc.})
  - Less memory hungry (MPI runtime consumption, O(P) data structures, etc.)
  - Load/stores to access memory instead of message passing
  - P is reduced by constant C (#cores/process) for O(Px) communication patterns

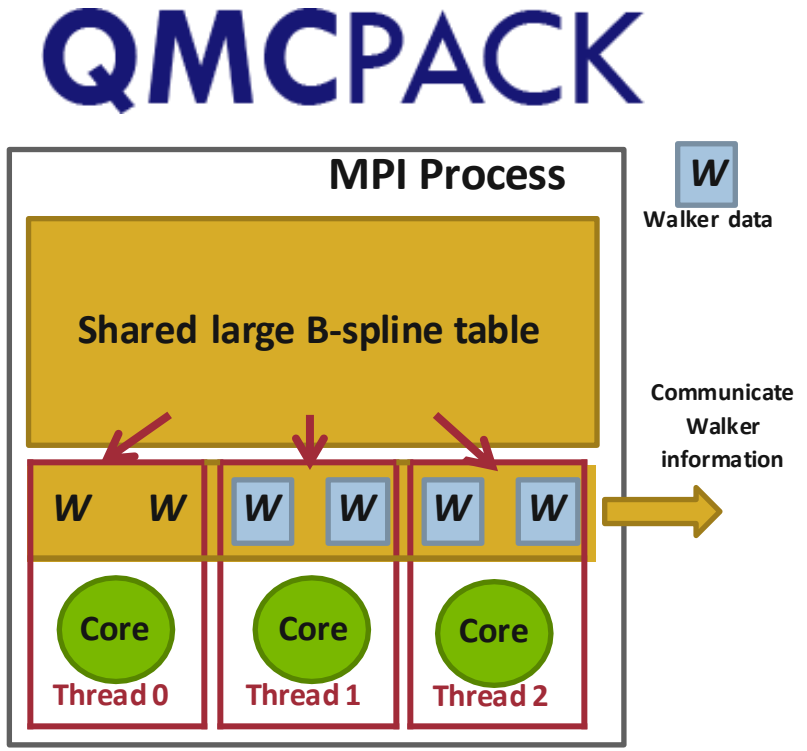
- Example 1: the Nek5000 team is working at the strong scaling limit

Nek5000



Why Hybrid MPI+X? Towards Strong Scaling (3/3)

- Example 2: Quantum Monte Carlo Simulation (QMCPACK)
  - Size of the physical system to simulate is bound by memory capacity [1]
  - Memory space dominated by large interpolation tables (typically several Giga Bytes of storage)
  - Threads are used to share those tables
  - Memory for communication buffers must be kept low to be allow simulation of larger and highly detailed simulations.

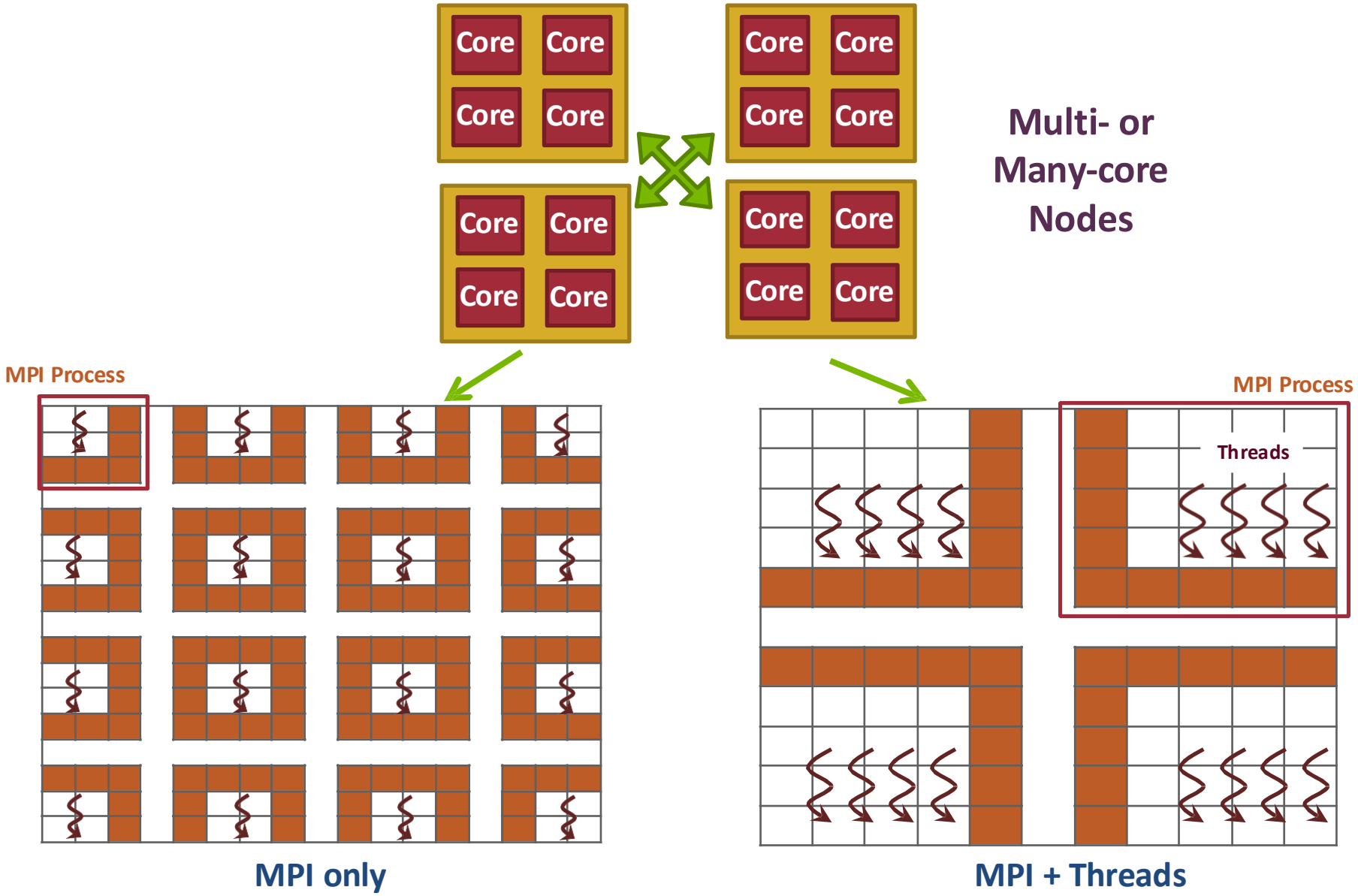


[1] Kim, Jeongnim, et al. "Hybrid algorithms in quantum Monte Carlo." Journal of Physics, 2012.

MPI Hybrid Programming: Threads

MPI Hybrid Programming: Threads

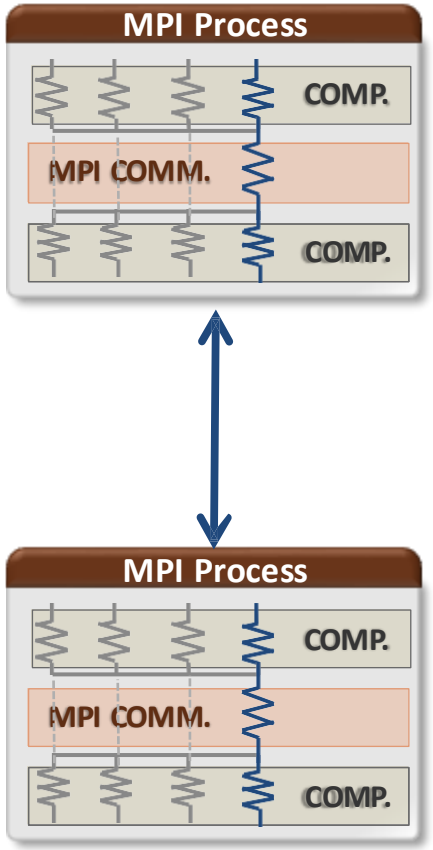
MPI + Threads: How To? (1/3)



MPI + Threads: How To? (2/3)

- MPI describes parallelism between *processes* (with separate address spaces)
- *Thread* parallelism provides a shared-memory model within a process
- OpenMP and Pthreads are common models
  - OpenMP provides convenient features for loop-level parallelism. Threads are created and managed by the compiler, based on user directives.
  - Pthreads provide more complex and dynamic approaches.

    Threads are created and managed explicitly by the user.



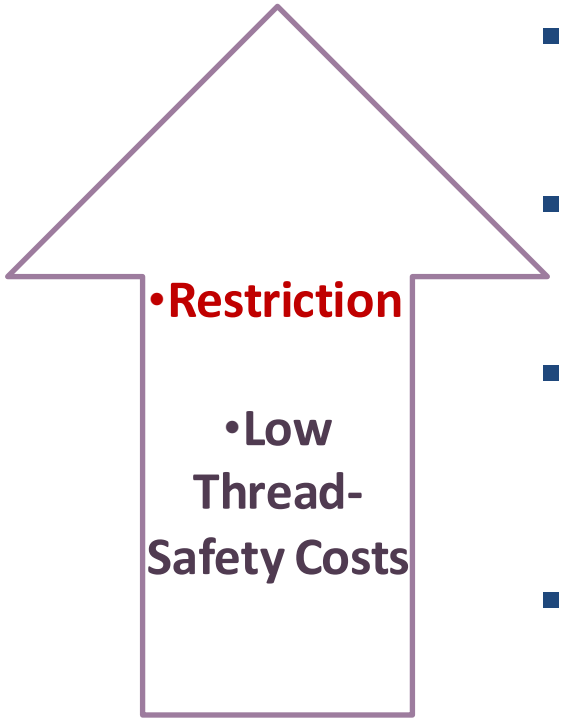
MPI + Threads: How To? (3/3)

MPI + Threads

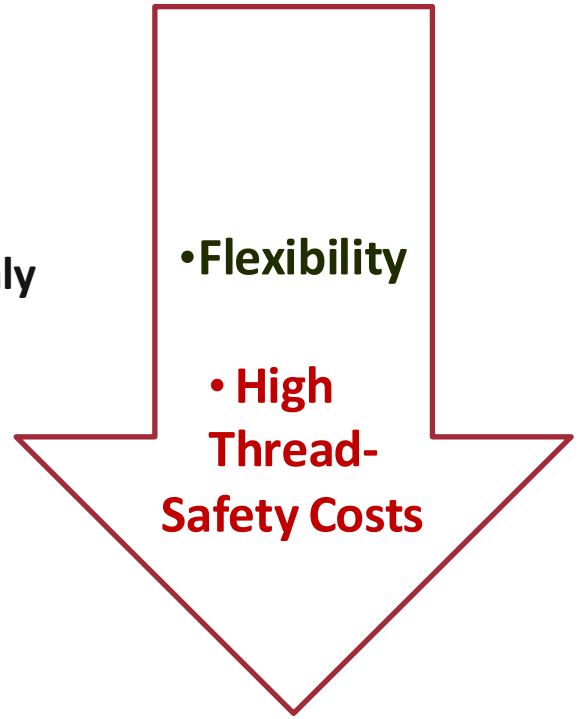
↓

Interoperability

Interoperation or thread levels:



- MPI_THREAD_SINGLE
  - No additional threads
- MPI_THREAD_FUNNELED
  - Master thread communication only
- MPI_THREAD_SERIALIZED
  - Threaded communication serialized
- MPI_THREAD_MULTIPLE
  - No restrictions



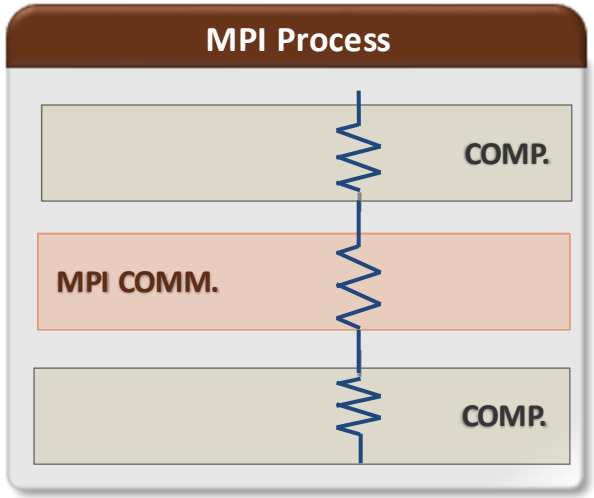
MPI's Four Levels of Thread Safety

- MPI defines four levels of thread safety -- these are commitments the application makes to the MPI
- Thread levels are in increasing order
  - If an application works in FUNNELED mode, it can work in SERIALIZED
- MPI defines an alternative to MPI_Init
  - MPI_Init_thread(requested, provided): *Application specifies level it needs; MPI implementation returns level it supports*

MPI_THREAD_SINGLE

- There are no additional user threads in the system
  - E.g., there are no thread parallel regions

```
int buf[100];
int main(int argc, char ** argv)
{
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    for (i = 0; i < 100; i++)
        compute(buf[i]);

    /* Do MPI stuff */

    MPI_Finalize();

    return 0;
}
```

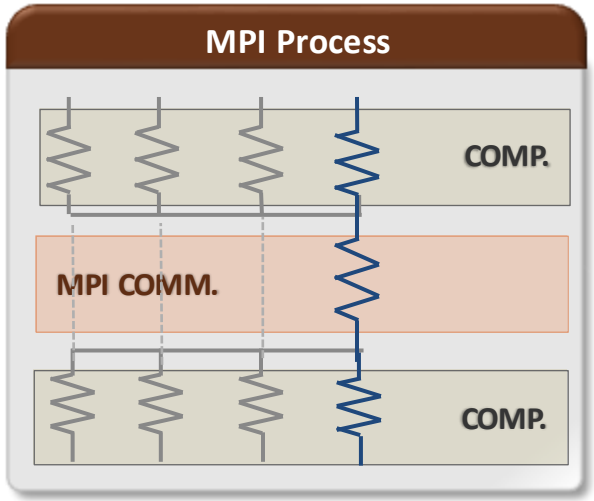


MPI_THREAD_FUNNELED

- All MPI calls are made by the master thread
  - Outside the thread parallel regions

```
int buf[100];
int main(int argc, char ** argv)
{
    int provided;

    MPI_Init_thread(&argc, &argv,
    MPI_THREAD_FUNNELED, &provided);
    if (provided < MPI_THREAD_FUNNELED)
        MPI_Abort(MPI_COMM_WORLD,1);

    for (i = 0; i < 100; i++)
        pthread_create(…,func,(void*)i);
    for (i = 0; i < 100; i++)
        pthread_join();

    /* Do MPI stuff */

    MPI_Finalize();
    return 0;
}
```

```
void* func(void* arg) {
    int i = (int)arg;
    compute(buf[i]);
}
```

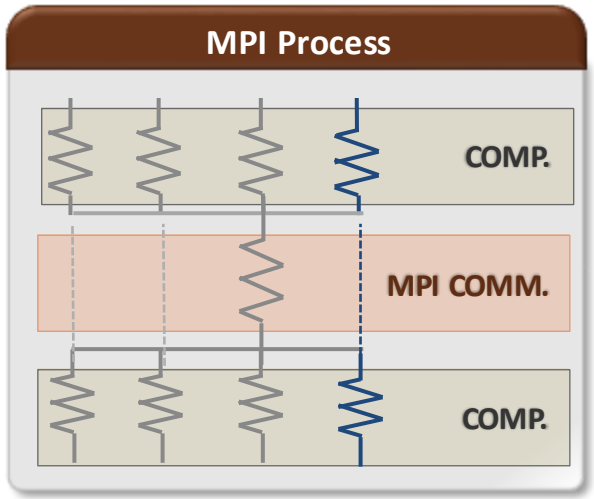


MPI_THREAD_SERIALIZED

- Only one thread can make MPI calls at a time
  - Protected by thread critical regions

```
int buf[100];
int main(int argc, char ** argv)
{
    int provided;
    pthread_mutex_t mutex;

    MPI_Init_thread(&argc, &argv,
    MPI_THREAD_SERIALIZED, &provided);
    if (provided < MPI_THREAD_SERIALIZED)
    MPI_Abort(MPI_COMM_WORLD,1);

    for (i = 0; i < 100; i++)
         pthread_create(…,func,(void*)i);
    for (i = 0; i < 100; i++)
         pthread_join();

    MPI_Finalize();
    return 0;
}
```

```
void* func(void* arg) {
    int i = (int)arg;
    compute(buf[i]);
    pthread_mutex_lock(&mutex);
    /* Do MPI stuff */
    pthread_mutex_unlock(&mutex);
}
```

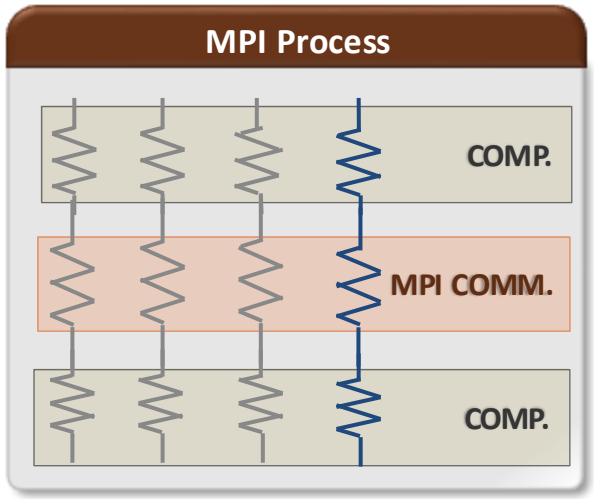


MPI_THREAD_MULTIPLE

- Any thread can make MPI calls any time (restrictions apply)

```
int buf[100];
int main(int argc, char ** argv)
{
    int provided;

    MPI_Init_thread(&argc, &argv,
    MPI_THREAD_MULTIPLE, &provided);
    if (provided < MPI_THREAD_MULTIPLE)
        MPI_Abort(MPI_COMM_WORLD,1);

    for (i = 0; i < 100; i++)
        pthread_create(…,func,(void*)i);
    for (i = 0; i < 100; i++)
        pthread_join();

    MPI_Finalize();
    return 0;
}
```

```
void* func(void* arg) {
    int i = (int)arg;
    compute(buf[i]);

    /* Do MPI stuff */
}
```



Threads and MPI

- An implementation is not required to support levels higher than MPI_THREAD_SINGLE; that is, an implementation is not required to be thread safe
- A fully thread-safe implementation will support MPI_THREAD_MULTIPLE
- A program that calls MPI_Init (instead of MPI_Init_thread) should assume that only MPI_THREAD_SINGLE is supported
  - MPI Standard *mandates* MPI_THREAD_SINGLE for MPI_Init
- *A threaded MPI program that does not call MPI_Init_thread is an incorrect program (common user error we see)*

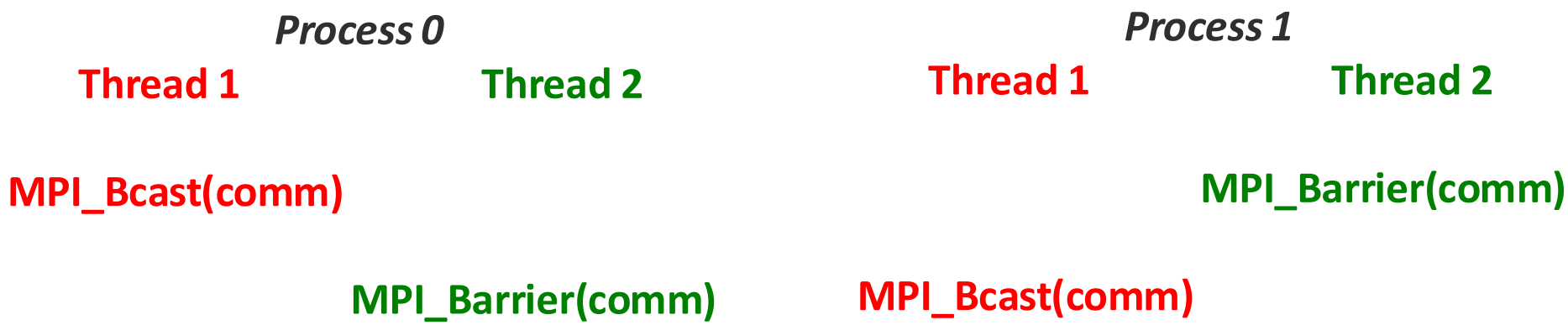
MPI Semantics and MPI_THREAD_MULTIPLE

- *Ordering:* When multiple threads make MPI calls concurrently, the outcome will be as if the calls executed sequentially in some (any) order
  - Ordering is maintained within each thread
  - User must ensure that collective operations on the same communicator, window, or file handle are correctly ordered among threads
    - E.g., cannot call a broadcast on one thread and a reduce on another thread on the same communicator
  - It is the user's responsibility to prevent races when threads in the same application post conflicting MPI calls
    - E.g., accessing an info object from one thread and freeing it from another thread
- *Progress:* Blocking MPI calls will block only the calling thread and will not prevent other threads from running or executing MPI functions

Ordering in MPI_THREAD_MULTIPLE: Incorrect Example with Collectives

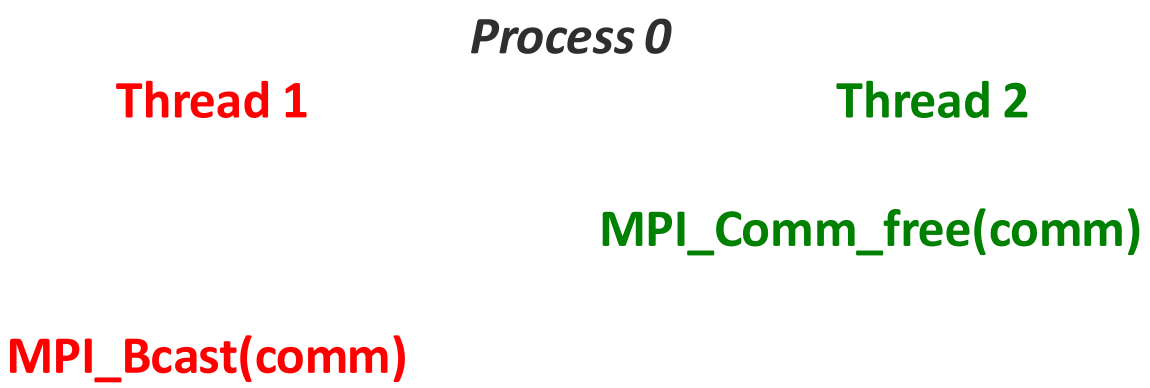
	Process 0	*Process 1*
Thread 0	MPI_Bcast(comm) MPI_Bcast(comm)	
Thread 1	MPI_Barrier(comm)	MPI_Barrier(comm)

Ordering in MPI_THREAD_MULTIPLE: Incorrect Example with Collectives



- P0 and P1 can have different orderings of Bcast and Barrier
- Here the user must use some kind of synchronization to ensure that either thread 1 or thread 2 gets scheduled first on both processes
- Otherwise a broadcast may get matched with a barrier on the same communicator, which is not allowed in MPI

Ordering in MPI_THREAD_MULTIPLE: Incorrect Example with Object Management



- The user has to make sure that one thread is not using an object while another thread is freeing it
  - This is essentially an ordering issue; the object might get freed before it is used

Blocking Calls in MPI_THREAD_MULTIPLE: Correct Example

	Process 0	*Process 1*
Thread 1	MPI_Recv(src=1)	MPI_Recv(src=0)
Thread 2	MPI_Send(dst=1)	MPI_Send(dst=0)

- An implementation must ensure that the above example never deadlocks for any ordering of thread execution
- That means the implementation cannot simply acquire a thread lock and block within an MPI function. It must release the lock to allow other threads to make progress.

The Current Situation

- All MPI implementations support MPI_THREAD_SINGLE
- They probably support MPI_THREAD_FUNNELED even if they don't admit it.
  - Does require thread-safety for some system routines (e.g. malloc)
  - On most systems `-pthread` will guarantee it (OpenMP implies `-pthread`)
- Many (but not all) implementations support THREAD_MULTIPLE
  - Hard to implement efficiently though (thread synchronization issues)
- Bulk-synchronous OpenMP programs (loops parallelized with OpenMP, communication between loops) only need FUNNELED
  - So don't need "thread-safe" MPI for many hybrid programs
  - But watch out for Amdahl's Law!

Hybrid Programming: Correctness Requirements

- Hybrid programming with MPI+threads does not do much to reduce the complexity of thread programming
  - Your application still has to be a correct multi-threaded application
  - On top of that, you also need to make sure you are correctly following MPI semantics
- Many commercial debuggers offer support for debugging hybrid MPI+threads applications (mostly for MPI+Pthreads and MPI+OpenMP)

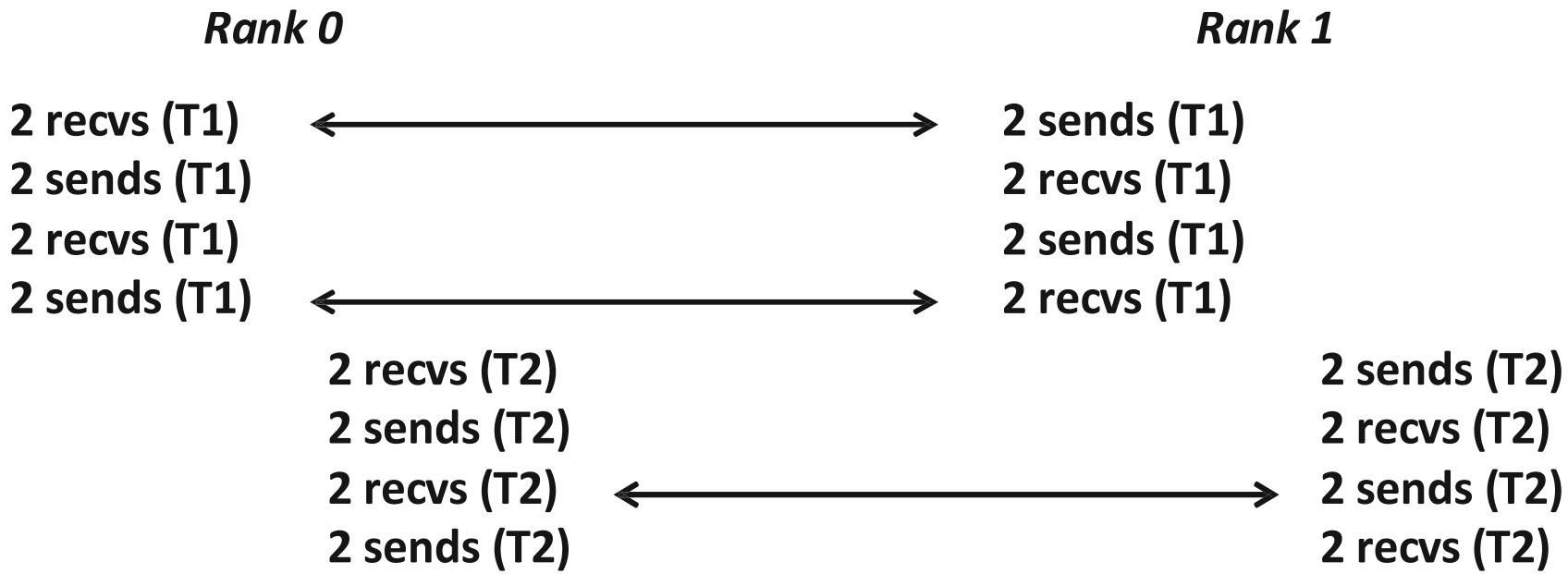
An Example we encountered

- We received a bug report about a very simple multithreaded MPI program that hangs
- Run with 2 processes
- Each process has 2 threads
- Both threads communicate with threads on the other process as shown in the next slide
- We spent several hours trying to debug MPICH before discovering that the bug is actually in the user's program ☹

2 Processes, 2 Threads (each thread executes this code)

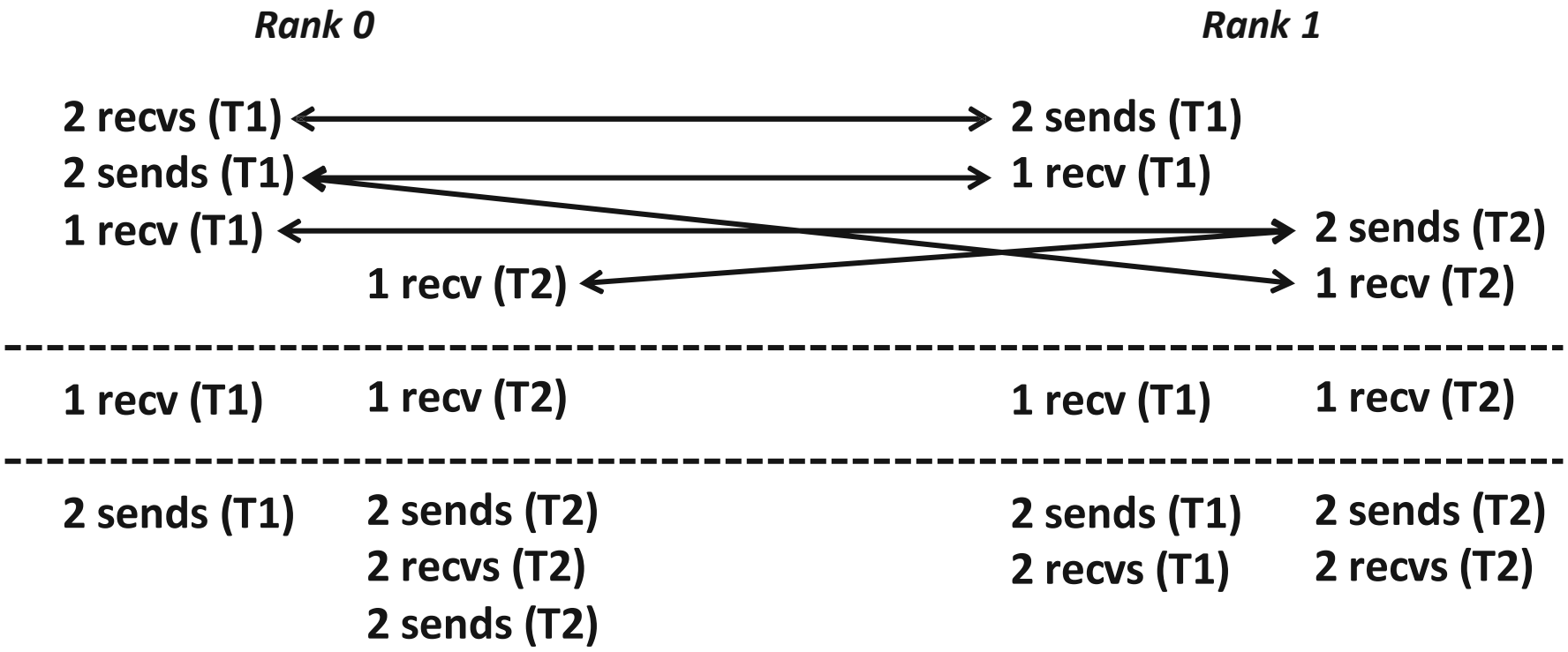
```
if (rank == 1) {
    MPI_Send(NULL, 0, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    MPI_Send(NULL, 0, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    MPI_Recv(NULL, 0, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &stat);
    MPI_Recv(NULL, 0, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &stat);

    MPI_Send(NULL, 0, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    MPI_Send(NULL, 0, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    MPI_Recv(NULL, 0, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &stat);
    MPI_Recv(NULL, 0, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &stat);
} else {  /* rank == 0 */
    MPI_Recv(NULL, 0, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &stat);
    MPI_Recv(NULL, 0, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &stat);
    MPI_Send(NULL, 0, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
    MPI_Send(NULL, 0, MPI_CHAR, 1, 0, MPI_COMM_WORLD);

    MPI_Recv(NULL, 0, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &stat);
    MPI_Recv(NULL, 0, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &stat);
    MPI_Send(NULL, 0, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
    MPI_Send(NULL, 0, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
}
```

Intended Ordering of Operations



- Every send matches a receive on the other rank

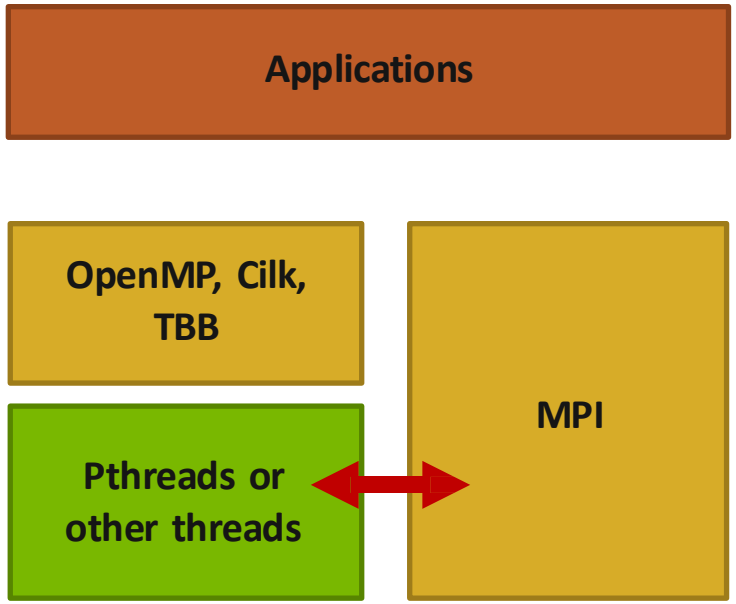
Possible Ordering of Operations in Practice



- Because the MPI operations can be issued in an arbitrary order across threads, all threads could block in a RECV call

MPI+OpenMP correctness semantics

- MPI only specifies interoperability with threads, not with OpenMP (or any other high-level programming model using threads)
  - OpenMP iterations need to be carefully mapped to which thread executes them (some schedules in OpenMP make this harder)
- For OpenMP tasks, the general model to use is that an OpenMP thread can execute one or more OpenMP tasks
  - An MPI blocking call should be assumed to block the entire OpenMP thread, so other tasks might not get executed



OpenMP threads: MPI blocking Calls (1/2)

```
int main(int argc, char ** argv)
{
    MPI_Init_thread(NULL, NULL, MPI_THREAD_MULTIPLE, &provided);

#pragma omp parallel for
    for (i = 0; i < 100; i++) {
        if (i % 2 == 0)
            MPI_Send(.., to_myself, ..);
        else
            MPI_Recv(.., from_myself, ..);
    }

    MPI_Finalize();

    return 0;
}
```

Iteration to OpenMP thread mapping needs to explicitly be handled by the user; otherwise, OpenMP threads might all issue the same operation and deadlock

OpenMP threads: MPI blocking Calls (2/2)

```
int main(int argc, char ** argv)
{
    MPI_Init_thread(NULL, NULL, MPI_THREAD_MULTIPLE, &provided);

#pragma omp parallel
{
    assert(omp_get_num_threads() > 1)
    #pragma omp for schedule(static, 1)
    for (i = 0; i < 100; i++) {
        if (i % 2 == 0)
            MPI_Send(.., to_myself, ..);
        else
            MPI_Recv(.., from_myself, ..);
    }
}
    MPI_Finalize();

    return 0;
}
```

Either explicit/careful mapping of iterations to threads, or using nonblocking versions of send/recv would solve this problem

OpenMP tasks: MPI blocking Calls (1/5)

```
int main(int argc, char ** argv)
{
     MPI_Init_thread(NULL, NULL, MPI_THREAD_MULTIPLE, &provided);

#pragma omp parallel
{
   #pragma omp for
   for (i = 0; i < 100; i++) {
      #pragma omp task
      {
        if (i % 2 == 0)
          MPI_Send(.., to_myself, ..);
        else
          MPI_Recv(.., from_myself, ..);
      }
   }
}
     MPI_Finalize();
     return 0;
}
```

This can lead to deadlocks. No ordering or progress guarantees in OpenMP task scheduling should be assumed; a blocked task blocks it's thread and tasks can be executed in any order.

OpenMP tasks: MPI blocking Calls (2/5)

```
int main(int argc, char ** argv)
{
    MPI_Init_thread(NULL, NULL, MPI_THREAD_MULTIPLE, &provided);

#pragma omp parallel
{
   #pragma omp taskloop
   for (i = 0; i < 100; i++) {
      if (i % 2 == 0)
        MPI_Send(.., to_myself, ..);
      else
        MPI_Recv(.., from_myself, ..)
   }
}
    MPI_Finalize();
    return 0;
}
```

Same problem as before.

OpenMP tasks: MPI blocking Calls (3/5)

```
int main(int argc, char ** argv)
{
     MPI_Init_thread(NULL, NULL, MPI_THREAD_MULTIPLE, &provided);

#pragma omp parallel
{
    #pragma omp taskloop
    for (i = 0; i < 100; i++) {
       MPI_Request req;
       if (i % 2 == 0)
          MPI_Isend(.., to_myself, .., &req);
       else
          MPI_Irecv(.., from_myself, .., &req);
       MPI_Wait(&req, ..);
    }
}
     MPI_Finalize();
     return 0;
}
```

Using nonblocking operations but with MPI_Wait inside the task region does not solve the problem

OpenMP tasks: MPI blocking Calls (4/5)

```
int main(int argc, char ** argv)
{
     MPI_Init_thread(NULL, NULL, MPI_THREAD_MULTIPLE, &provided);

#pragma omp parallel
{
   #pragma omp taskloop
   for (i = 0; i < 100; i++) {
            MPI_Request req; int done = 0;
         if (i % 2 == 0)
            MPI_Isend(.., to_myself, .., &req);
         else
            MPI_Irecv(.., from_myself, .., &req);
         while (!done) {
            #pragma omp taskyield
            MPI_Test(&req, &done, ..);
         }
      }
   }
}
     MPI_Finalize();
     return 0;
}
```

Still incorrect; taskyield does not guarantee a task switch

OpenMP tasks: MPI blocking Calls (5/5)

```
int main(int argc, char ** argv)
{
     MPI_Init_thread(NULL, NULL, MPI_THREAD_MULTIPLE, &provided);
     MPI_Request req[100];

#pragma omp parallel
{
   #pragma omp taskloop
   for (i = 0; i < 100; i++) {
      if (i % 2 == 0)
         MPI_Isend(.., to_myself, .., &req[i]);
      else
         MPI_Irecv(.., from_myself, .., &req[i]);
   }
}
     MPI_Waitall(100, req, ..);
     MPI_Finalize();
     return 0;
}
```

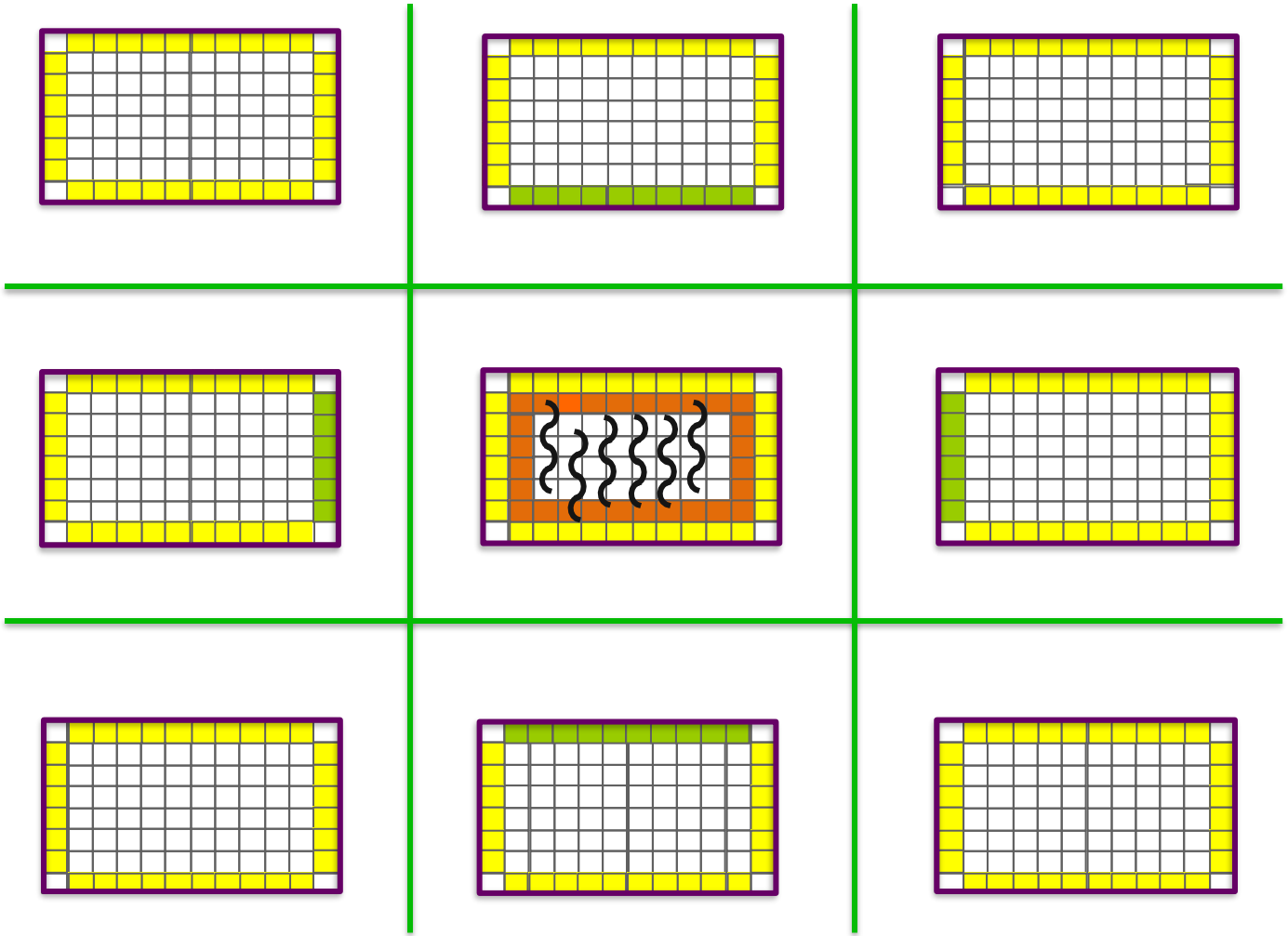
Correct example. Each task is nonblocking.

Ordering in MPI_THREAD_MULTIPLE: Incorrect Example with RMA

```
int main(int argc, char ** argv)
{
    /* Initialize MPI and RMA window */

#pragma omp parallel for
    for (i = 0; i < 100; i++) {
        target = rand();
        MPI_Win_lock(MPI_LOCK_EXCLUSIVE, target, 0, win);
        MPI_Put(..., win);
        MPI_Win_unlock(target, win);
    }

    /* Free MPI and RMA window */

    return 0;
}
```

Different threads can lock the same process causing multiple locks to the same target before the first lock is unlocked

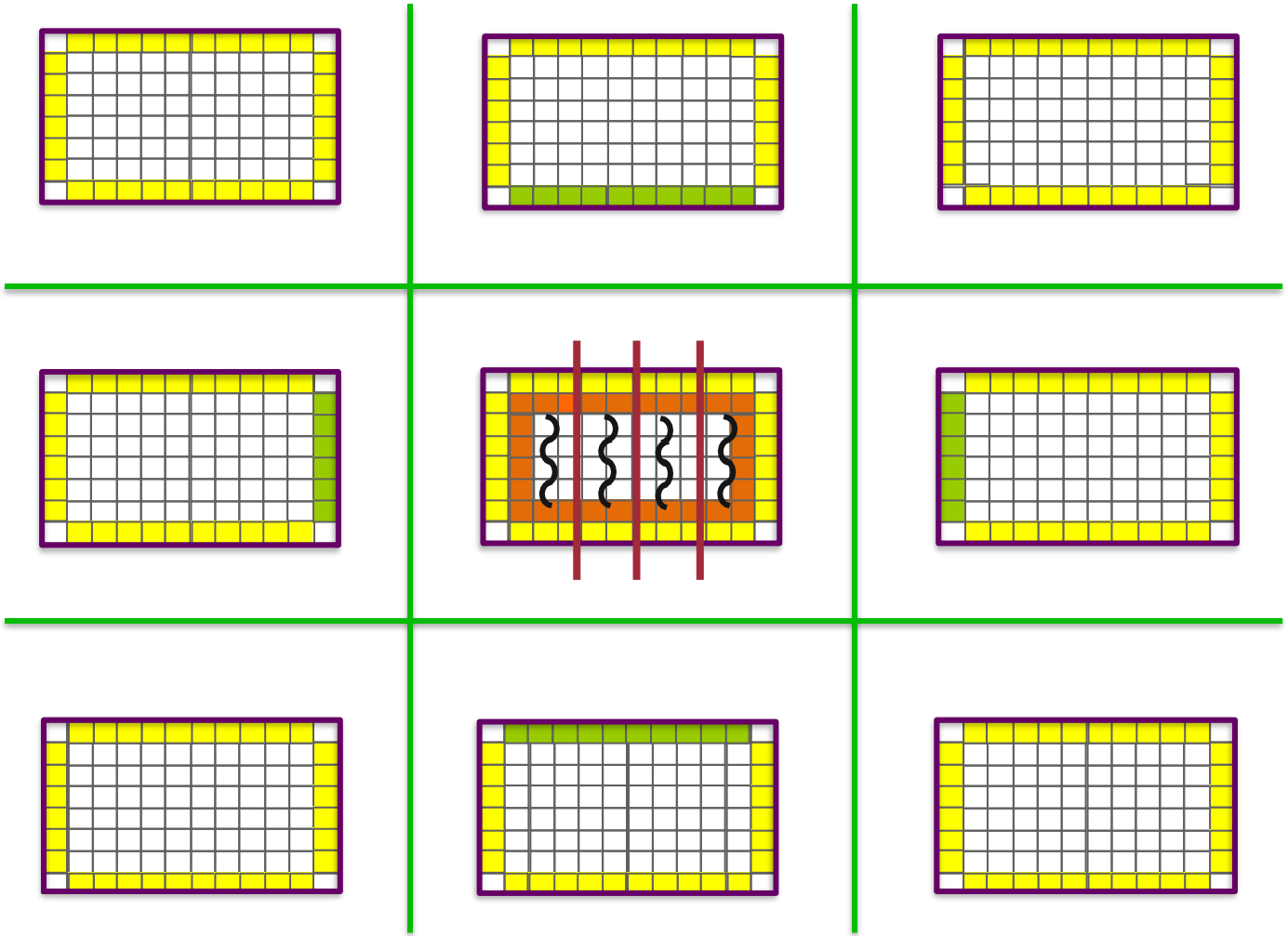
Exercise 1: Stencil in Funneled mode (1/2)



Exercise 1: Stencil in Funneled mode (2/2)

- Parallelize computation (OpenMP parallel for)
- Main thread does all communication
- *Start from derived_datatype/stencil.c*
- *Solution available in threads/stencil_funneled.c*

Exercise 2: Stencil in Multiple mode (1/2)



Exercise 2: Stencil in Multiple mode (2/2)

- Divide the process memory among OpenMP threads
- Each thread responsible for communication and computation
- *Start from threads/stencil_funneled.c*
- *Solution available in threads/stencil_multiple.c*

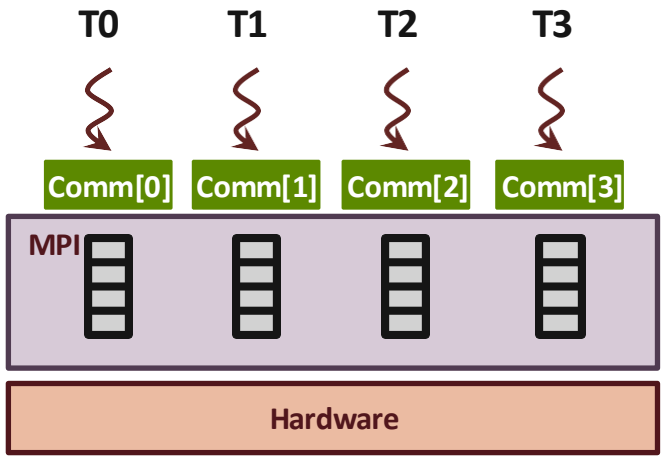
MPI+threads performance recommendations

MPI+threads performance recommendations

Recommendation: Maximize independence between threads with communicators

- Each thread accesses to a different communicator
  - Each communicator may be associated with isolated resource in an MPI implementation

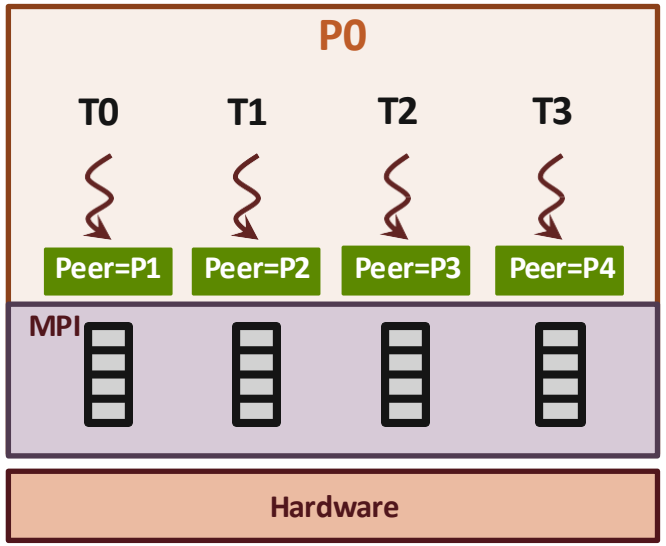
```
MPI_Comm *comms;
int nthreads = omp_get_num_threads();
comms = malloc(sizeof(MPI_Comm) * nthreads);

for (i = 0; i < nthreads; i++)
    MPI_Comm_dup(MPI_COMM_WORLD, &comms[i]);

#pragma omp parallel
{
   int tid = omp_get_thread_num();
   #pragma omp taskloop
   for (i = 0; i < 100; i++)
        MPI_Isend(.., comm[tid], &req[i]);}
}
MPI_Waitall(100, req, ..);
```



Recommendation: Maximize independence between threads with ranks or tags (1/2)

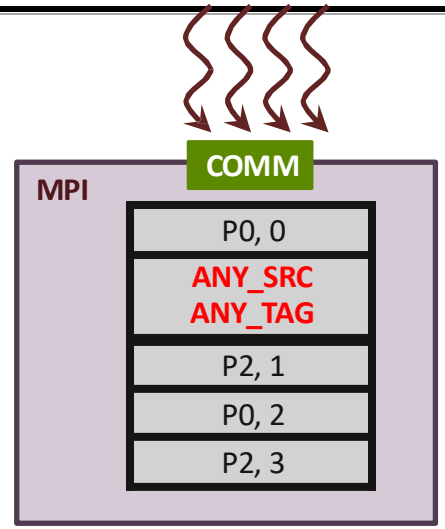
- Each thread communicates with different peer_rank or tag
  - MPI may assign isolated resource for different set of [peer_rank + tag]

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    #pragma omp taskloop
    for (i = 0; i < 100; i++)
          MPI_Isend(.., peer_ranks[tid], tid,
                    comm, &req[i]);}
}
MPI_Waitall(100, req, ..);
```



Recommendation: Maximize independence between threads with ranks and tags (2/2)

- Threads have to match all receive messages in sequential (e.g., a single receive-queue) order
  - Ensure ordering of message matching
  - With rank-based parallelization, only the sender can be parallelized
  - With tag-based parallelization, neither the sender nor the receiver can be parallelized
- Let MPI know if you do not use wildcard receive
  - Info hints mpi_assert_no_any_source, mpi_assert_no_any_tag (already proposed to MPI standard)
  - MPI can get rid of the single receive-queue for the communicator

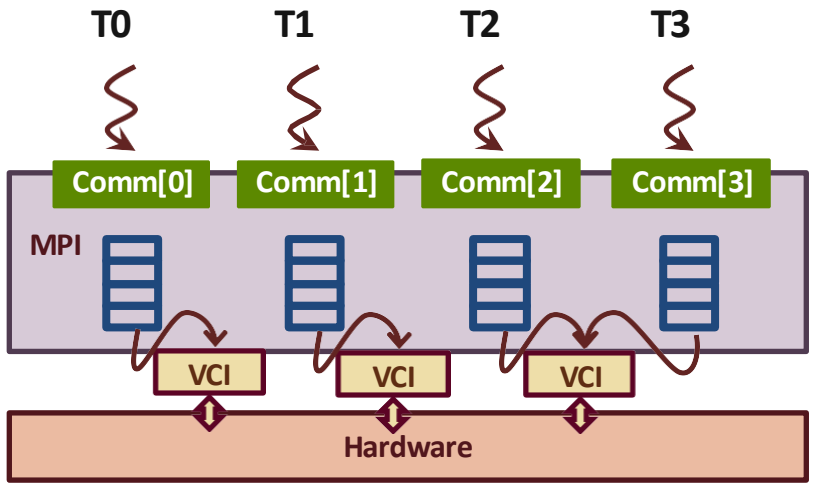



```
MPI_Info info;
info = MPI_Info_create();
MPI_Info_set(info,
   "mpi_assert_no_any_source", "true");
MPI_Comm_set_info(comm, info);
MPI_Info_free(&info);
/* Communicate without ANY_SOURCE */
```

Recommendation: Try to parallelize based on communicators; if you cannot, try ranks (with info); if you cannot, try tags (with info)

Possible Optimizations MPI libraries can do

- Virtual Communication Interface (VCI)
  - Each VCI abstracts a set of network/shared-memory resources
  - Some networks support multiple VCIs: InfiniBand contexts, scalable endpoints over Intel Omni-Path
  - Traditional MPI implementation uses single VCI
    - Serializes all traffic
    - Does not fully exploit network hardware resources
- Utilizing multiple VCIs to maximize independence in communication
  - Separate VCIs per communicator or per RMA window
  - Distribute traffic between VCIs with respect to ranks, tags, and generally out-of-order communication
  - M-N mapping between Work-Queues and VCIs



Exercise 3: Stencil with Independent Communicators

- Divide the process memory among OpenMP threads
- Each thread responsible for communication and computation
- Each thread uses a different communicator
- *Start from threads/stencil_multiple.c*
- *Solution available in threads/stencil_multiple_ncomms.c*

Section Summary

- Hybrid MPI + "X" is a promising approach for large scale programming (e.g., MPI+Threads)
  - Less memory consumption
  - More efficient on-node data movement (load/store)
- MPI thread safety: SINGLE, FUNNELED, SERIALIZED, MULTIPLE
- Use MPI_Init_thread for threaded programs (i.e., not SINGLE)
- THREAD_MULTIPLE ordering & progress semantics
- Always maximize independence between threads in your program
  - Independent communicators, no wildcard, independent peer_ranks and tags

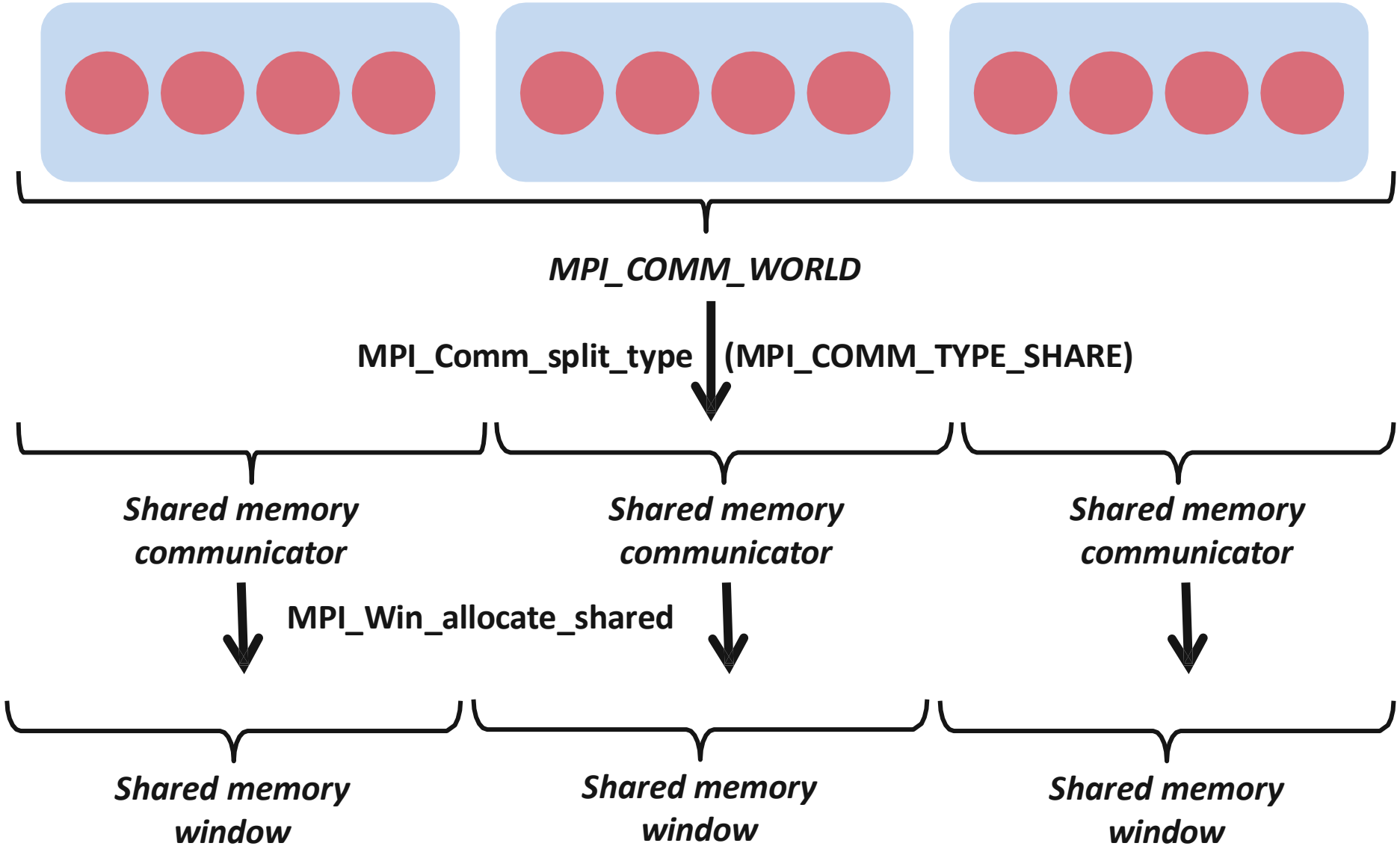
MPI Hybrid Programming: Shared Memory

MPI Hybrid Programming: Shared Memory

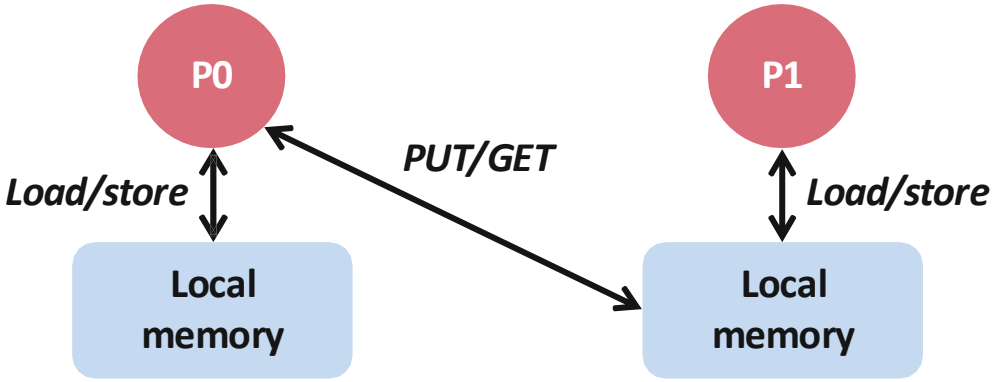
Hybrid Programming with Shared Memory

- MPI-3 allows different processes to allocate shared memory through MPI
  - MPI_Win_allocate_shared
- Uses many of the concepts of one-sided communication
- Applications can do hybrid programming using MPI or load/store accesses on the shared memory window
- Other MPI functions can be used to synchronize access to shared memory regions
- Can be simpler to program than threads

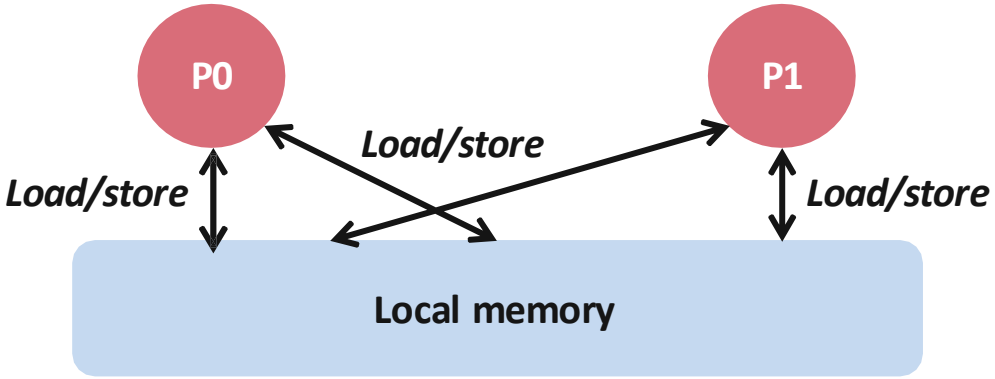
Creating Shared Memory Regions in MPI



Regular RMA windows vs. Shared memory windows



Traditional RMA windows



Shared memory windows

- Shared memory windows allow application processes to directly perform load/store accesses on all of the window memory
  - E.g., x[100] = 10
- All of the existing RMA functions can also be used on such memory for more advanced semantics such as atomic operations
- Can be very useful when processes want to use threads only to get access to all of the memory on the node
  - You can create a shared memory window and put your shared data

MPI_COMM_SPLIT_TYPE

```
MPI_Comm_split_type(MPI_Comm comm, int split_type, int key, MPI_Info info,
                    MPI_Comm *newcomm)
```

- Create a communicator where processes “share a property”
  - Properties are defined by the “split_type”
- Arguments:
  - comm - input communicator (handle)
  - split_type - property of the partitioning (integer)
  - key - rank assignment ordering (nonnegative integer)
  - info - info argument (handle)
  - newcomm - output communicator (handle)

MPI_WIN_ALLOCATE_SHARED

```
MPI_Win_allocate_shared(MPI_Aint size, int disp_unit, MPI_Info info, MPI_Comm comm,
                        void *baseptr, MPI_Win *win)
```

- Create a remotely accessible memory region in an RMA window
  - Data exposed in a window can be accessed with RMA ops or load/store
- Arguments:
  - size - size of local data in bytes (nonnegative integer)
  - disp_unit - local unit size for displacements, in bytes (positive integer)
  - info - info argument (handle)
  - comm - communicator (handle)
  - baseptr - pointer to exposed local data
  - win - window (handle)

Shared Arrays with Shared memory windows

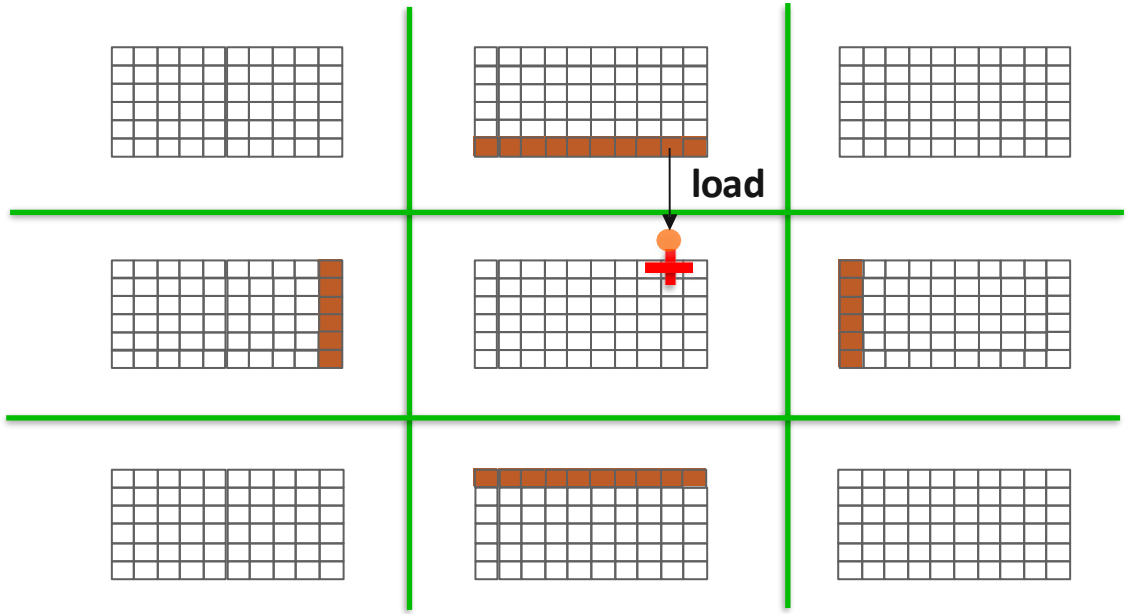
```
int main(int argc, char ** argv)
{
    int buf[100];

    MPI_Init(&argc, &argv);
    MPI_Comm_split_type(..., MPI_COMM_TYPE_SHARED, .., &comm);
    MPI_Win_allocate_shared(comm, ..., &win);

    MPI_Win_lockall(win);

    /* copy data to local part of shared memory */
    MPI_Win_sync(win);

    /* use shared memory */

    MPI_Win_unlock_all(win);

    MPI_Win_free(&win);
    MPI_Finalize();
    return 0;
}
```

Memory allocation and placement

- Shared memory allocation does not need to be uniform across processes
  - Processes can allocate a different amount of memory (even zero)
- The MPI standard does not specify where the memory would be placed (e.g., which physical memory it will be pinned to)
  - Implementations can choose their own strategies, though it is expected that an implementation will try to place shared memory allocated by a process "close to it"
- The total allocated shared memory on a communicator is contiguous by default
  - Users can pass an info hint called "noncontig" that will allow the MPI implementation to align memory allocations from each process to appropriate boundaries to assist with placement

Exercise4: Stencil with Shared Memory

- Message passing model requires ghost-cells to be explicitly communicated to neighbor processes
- In the shared-memory model, there is no communication. Neighbors directly access your data.
- *Start from rma/stencil_lock_put.c*
- *Solution available in shared_mem/stencil.c*



Threads vs. Process Shared Memory

- It depends on the application, target machine, and MPI implementation
- When should I use process shared memory?
  - The only resource that needs sharing is memory
  - Few allocated objects need sharing (easy to place them in a public shared region)
- When should I use threads?
  - More than memory resources need sharing (e.g., TLB)
  - Many application objects require sharing
  - Application computation structure can be easily parallelized with high-level OpenMP loops

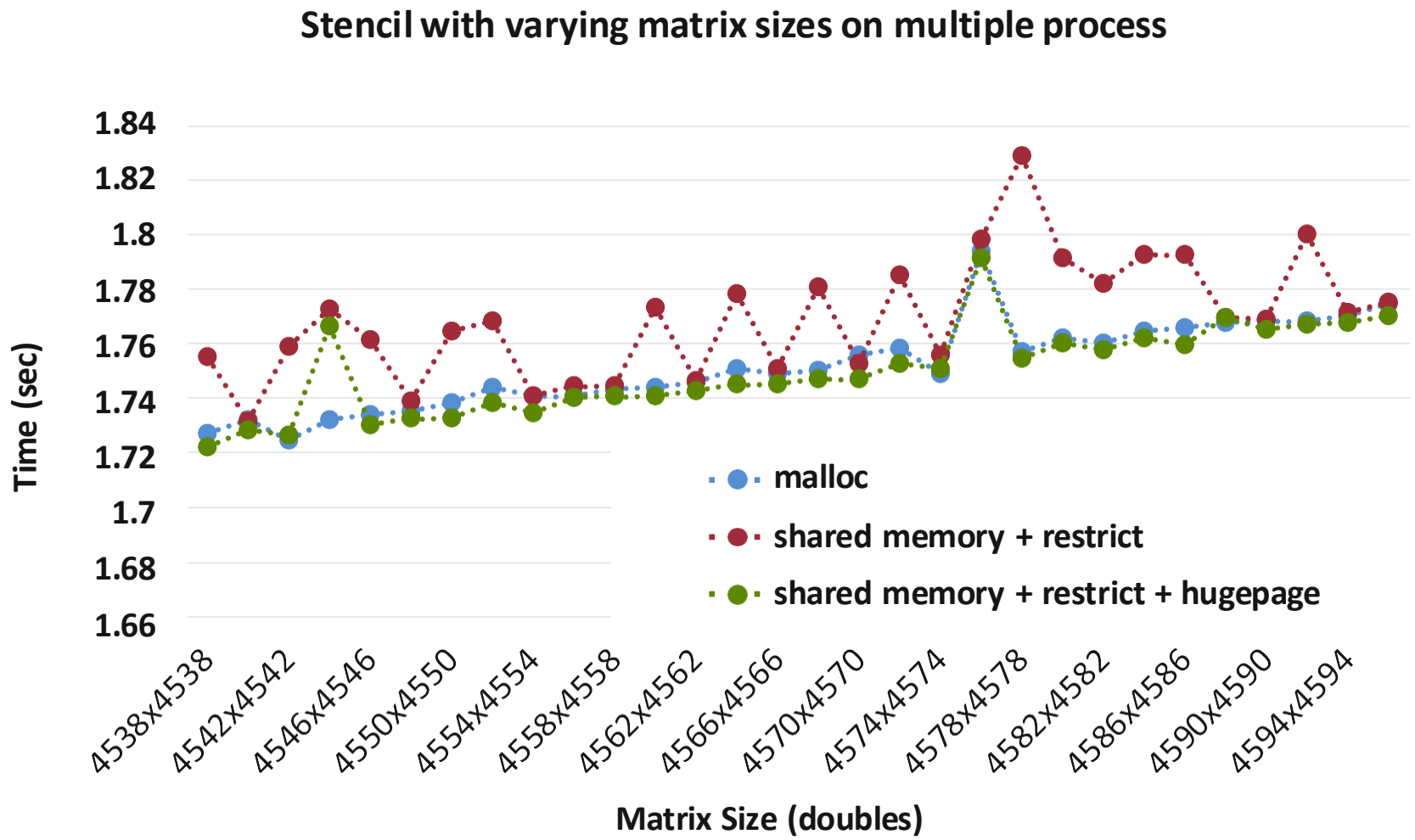
Shortcomings: OS Interference (1/3)

- OS treats regular memory allocation and shared memory allocation differently
  - Single process memory allocation internally does an anonymous mmap
    - The OS "likes" this type of memory allocations and assigns large pages (2MB) to back such memory
  - Multi-process memory allocation (shared memory) internally does a file-backed mmap
    - The OS assigns regular sized pages (4KB) to back such memory

Shortcomings: OS Interference (2/3)

- Solutions:
  - Idea is to modify the OS to give us huge pages for shared memory
  - System settings (enable hugepage-backed ramfs):
    - the hugepage number in /proc/sys/vm/nr_hugepages
    - the group id of huge page in /proc/sys/vm/hugetlb_shm_group
    - These two can also be done by the sysctl command by the root, or they can be set in the configuration file /etc/sysctl.conf.
  - Within the MPI implementation, allocate the file backing the shared memory on “hugetlbfs”

Shortcomings: OS Interference (3/3)



Shortcomings: Restricted Allocation Methods

- In MPI-3 shared memory, memory allocation is restrictive
  - Allocation has to be done using the MPI call
  - Cannot use the plethora of other memory allocation libraries out there, e.g., cannot allocate aligned memory (important for vectorization)
- With threads, most of those other memory allocation techniques are directly usable

Section Summary

- Hybrid programming with MPI-3 shared memory
  - MPI_Comm_split_type + MPI_Win_allocate_shared
  - Can use MPI functions (e.g., RMA operations) or direct load/store to access the shared memory
- MPI + Process shared memory vs. MPI + Threads
  - Shared memory: only share memory and few allocated objects
  - Threads: more resource sharing, more objects sharing, computation structure fits high-level thread-based parallelism (e.g., OpenMP loops)
- Some performance optimizations:
  - Use restrict for your shared memory pointer
  - Enable hugepage

MPI Hybrid Programming: Accelerators

MPI + CUDA Programming

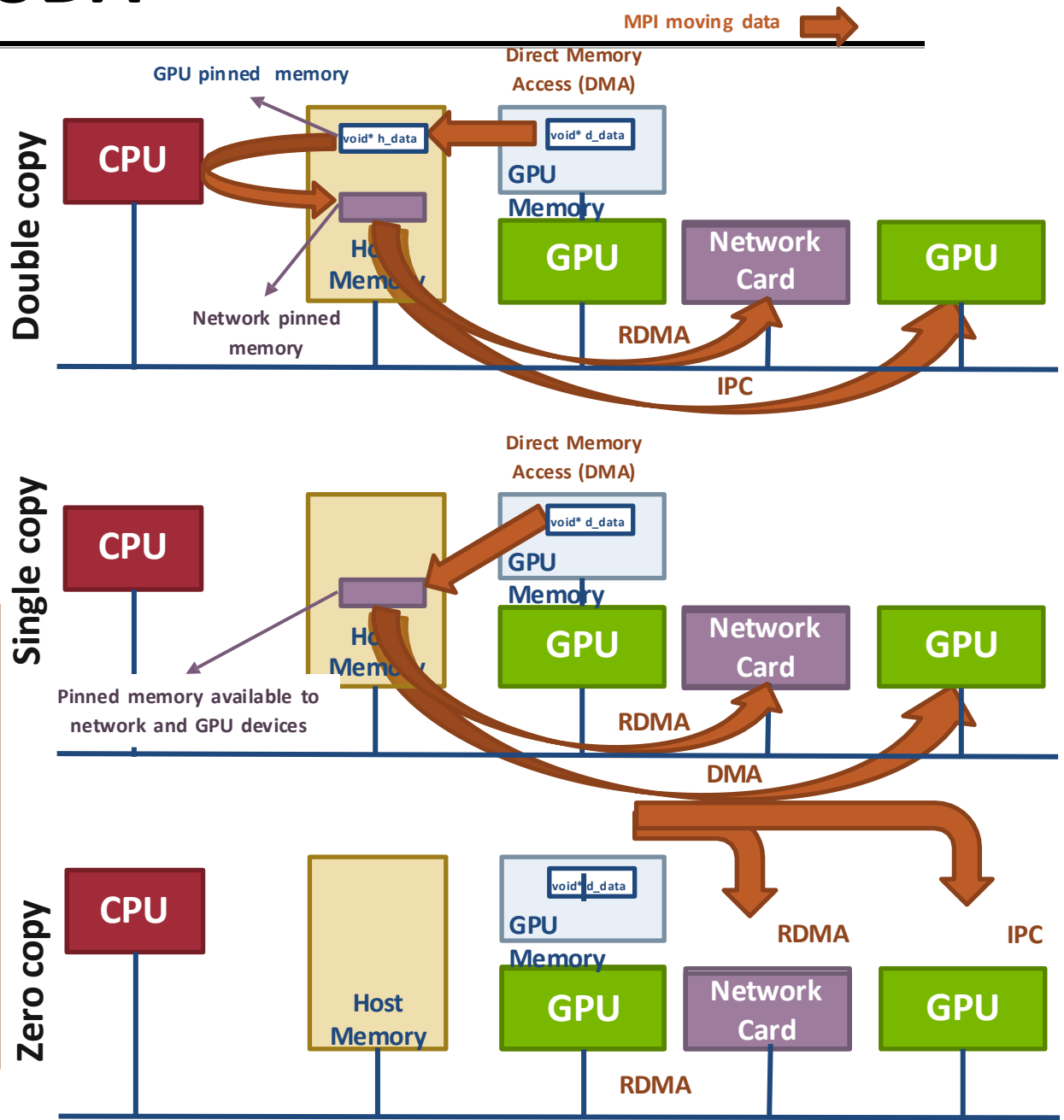
Overview

- General-purpose parallel computing platform and programming model released by NVIDIA in 2006
- Provides a user library (*libcuda*), runtime (*libcudart*), device drivers and C/C++ compiler (NVCC)
- Programming language extensions for C/C++
  - Define C functions to run on the GPU (kernels) using the __global__ declaration specifier
  - Kernels can be launched with different number of threads using the <<<...>>> execution syntax
  - Each thread executing the kernel is given a unique thread ID accessible from inside the kernel using the built-in threadIdx variable
- Support other languages such as FORTRAN

```
/* Kernel definition */
__global__ void gpu_kernel(double *in,
                           double *out)
{
  /* get indices from thread id */
  int i = threadIdx.x;
  int j = threadIdx.y;

  /* each thread performs work */
  out[i][j] = f(in, i, j);
}

int main()
{
  double *in_h, *out_h, *in_d, *out_d;
  in_h  = malloc(size);
  out_h = malloc(size);
  cudaMalloc(&in_d,  size);
  cudaMalloc(&out_d,  size);

  cudaMemcpy(in_d, in_h, size,
             cudaMemcpyHostToDevice);

  /* kernel invocation with N threads */
  gpu_kernel<<<2,N>>>(in_d,  out_d);

  cudaMemcpy(out_h, out_d, size,
             cudaMemcpyDeviceToHost);

  [...snip...]

  return 0;
}
```

GPU Direct Capabilities in CUDA

- GPU Direct 1.0 (Q2 2010) allowed single-copy communication
- GPU Direct 2.0 (2011) or GPU Direct IPC allowed for zero copy communication from GPU to GPU
- GPU Direct 3.0 (2013) or GPU Direct RDMA allowed for zero copy communication from GPU to network and back



- Need to ensure that the data being sent is ready

```
double *dev_buf;
cudaMalloc(&dev_buf, size);

if(my_rank == sender) {
    gpu_kernel<<<..>>>(dev_buf);   /* sync kernel */
    MPI_Isend(dev_buf, size, MPI_DOUBLE, receiver, 0,
   comm, req);
} else {
    MPI_Recv(dev_buf, size, MPI_DOUBLE, sender, 0,
   comm, &status);
    gpu_kernel<<<..>>>(dev_buf);
}
```

Hybrid MPI+CUDA example

```
int main(int argc, char ** argv)
{
    int buf[100];
    double *dev_buf;
    cudaMalloc(&dev_buf, size);

    MPI_Init(&argc, &argv);

    if(my_rank == sender) {
        gpu_kernel<<<..>>>(dev_buf);
        cudaDeviceSynchronize();
        MPI_Send(dev_buf, size, MPI_DOUBLE, receiver, 0, comm, req);
    } else {
        MPI_Recv(dev_buf, size, MPI_DOUBLE, sender, 0, comm, &status);
        gpu_kernel<<<..>>>(dev_buf);
        cudaDeviceSynchronize();
    }

    MPI_Finalize();
    return 0;
}
```

Summary

- Accelerators are becoming increasingly important in HPC
- MPI is playing its role in enabling the usage of accelerators across distributed memory nodes
- The situation with MPI + GPU support is improving in both MPI implementations and in GPU hardware/software
  - For CUDA and ROCm P2P/RDMA support from GPU memory is enabled for contiguous datatypes through the UCX driver
  - For OpenCL/SYCL SVM can potentially enable similar optimizations as CUDA/ROCm but at the current state no such support is available and explicit data movement between host and device memory is still required
  - For OpenMP/OpenACC data movement is managed through directives but compilers can decide to leverage HMM